FastMACE: Efficient GPU Inference and Molecular Dynamics for Pretrained MACE Potentials

Fabian Zwingel
University of Bayreuth, Germany

Draft of 12 November 2026

Abstract

MACE potentials are accurate but costly to evaluate: a general implementation runs each layer as separate kernels, carries large tensors for every neighbour pair, and returns to the host at every molecular dynamics step. FastMACE runs *unchanged pretrained checkpoints* faster while keeping MACE's own PyTorch modules. We make three claims. First, with frozen parameters the first interaction can be reassociated so that the work per neighbour pair shrinks to a sum of radial features, and adjacent linear maps and the first symmetric contraction can be composed or generated ahead of time. Second, what remains can be streamed: radial networks become piecewise-polynomial tables, the second interaction and its derivative run without per-edge weight tensors, dense products run on tensor cores, and complete MD steps run on the GPU as captured graphs. Third, storing only what the second interaction reads, and releasing it along a wavefront, lets a tiled evaluator handle systems far beyond device memory. For the OMAT-0 medium model on 5,000 atoms of $SrTiO_3$, recorded Langevin steps take 3.4 ms on an NVIDIA H200 and 6.2 ms on an A100. In a short benchmark that starts from rest, a step took 5.7 ms on the H200 against 14.6 ms for Symmetrix-XL 0.1.2; runs with thermal initial velocities and neighbour rebuilds on a smaller GPU gave ratios of 2.1 and 3.1. On 18 rattled and strained test cells, force and stress errors stay within 1.3 times the float32 error of the reference implementation, under an acceptance limit of three. The evidence has gaps, which we state: the default configuration has no energy-conservation record, its packed forward table is qualified offline only, the largest capacity runs predate a coordinate fix, and no downstream property has been validated.

Keywords: machine-learned interatomic potentials, MACE, equivariant neural networks, GPU kernels, molecular dynamics, tensor cores

1 Introduction

A molecular dynamics (MD) simulation evaluates the same trained potential at every step, with fixed parameters and slowly changing geometry. A general neural-network implementation does not use that structure. For MACE [1], a force call expands radial weights for every neighbour pair, forms tensor products edge by edge, keeps the results for automatic differentiation, and returns to a host-side integrator and neighbour list before the next step.

FastMACE is a library that runs pretrained MACE checkpoints on NVIDIA GPUs with these costs removed where the mathematics allows it. Its unit of optimisation is the complete energy, force and MD-step calculation, not a single layer. No retraining, distillation or change of time step is involved.

Three claims.

1. **Reassociation with frozen parameters reduces the repeated edge work** (Section 4). In the first interaction the projection onto the channels moves outside the neighbour sum; adjacent linear maps are multiplied once; the first symmetric contraction becomes a generated program with shared products.
2. **Streamed execution reaches the measured speed under an explicit fidelity contract** (Section 5). Radial networks become tables, the second interaction and its reverse run without per-edge weights, dense products use tensor cores, and an MD step is two captured graphs. The contract is a numeric acceptance rule on forces, stresses and energies (Section 3.1).
3. **Reduced-state scheduling extends capacity at a stated cost in time and transfers** (Section 6). A tiled evaluator stores only what the second interaction reads, releases it along a wavefront, and can keep it in host memory.

Closest work. Symmetrix-XL [2], published on 1 October 2026, has the same aim. It converts the checkpoint into a native evaluator with kernels generated for the model, streams edges, replaces the radial networks by splines, and evaluates large systems tile by tile in three sweeps. It runs under LAMMPS and ASE, on CPUs and on many GPUs, and it reproduces published downstream benchmarks. FastMACE adopted three of its ideas (Appendix B) and differs in keeping MACE's PyTorch modules, in its own MD engine, in tensor-core arithmetic and in how the tiled path stores its state. We compare with it wherever we have a measurement (Section 8.4) and say where it is ahead (Section 10).

What the evidence does and does not cover. The paper describes one concrete case, the OMAT-0 medium model in float32. Its numbers come from separate adoption studies on different dates and revisions, not from one benchmark of a single configuration. The numerical contract is a sampled test on small cells. Four gaps bear on the headline numbers and are taken up where they arise: today's default configuration has no energy-conservation record (Section 7); its packed forward table is not covered by the setup test (Section 5.1); the comparison with Symmetrix-XL on the main benchmark is a short run from rest without neighbour rebuilds (Section 8.4); and the largest tiled runs used an earlier coordinate path (Section 6.2). Section 10 lists the experiments that would close them.

Several of the mechanisms were first proposed elsewhere: in the literature, in other inference engines, or in research reports produced by large language models at the author's request. Appendix B records the origin of each.

2 The calculation being accelerated

Model. MACE [1] builds on the atomic cluster expansion [3] and equivariant message passing [4, 5]. For positions R, cell H and species z the energy is

$$E(R,H,z) \;=\; \sum_i E^0_{z_i} \;+\; \sum_i \varepsilon_i(R,H,z) \;+\; E_{\text{pair}}(R,H,z), \qquad F_i = -\frac{\partial E}{\partial R_i}, \tag{1}$$

where ε_i is the learned site energy and E_{pair} a short-range repulsion. Stress follows from the strain derivative.

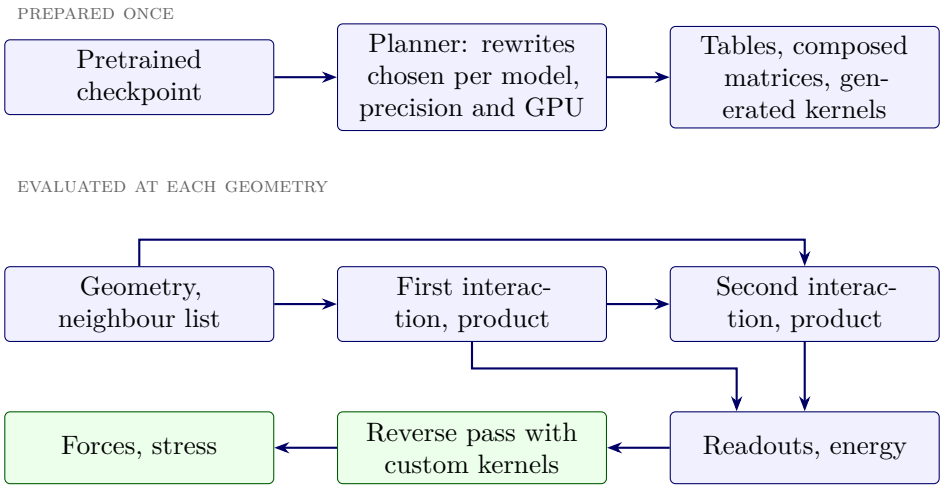


Figure 1: Structure of FastMACE. Preparation is allowed to be expensive because its products are reused. The repeated calculation is captured in CUDA graphs. A tiled evaluator (Section 6) reuses the same rewritten modules with a different storage schedule.

For a directed edge from sender j to receiver i let r_{ij} be its length and $Y_b(\hat{r}_{ij})$ its real spherical harmonics. An interaction forms the message

$$m^{(p)}_{i,o,c} = \sum_{j \in \mathcal{N}(i)} w_{p,c}(r_{ij}) \sum_{a,b} C^{(p)}_{oab}\, x_{j,a,c}\, Y_b(\hat{r}_{ij}), \tag{2}$$

with channel c, coupling path p, input and output components a and o, Clebsch–Gordan coefficients C, and radial weights w produced by a small network of the distance (and of the two species). A symmetric contraction then forms polynomial combinations of the message at each atom, equivariant linear maps mix channels, and readouts give ε_i.

The example model. All details below refer to OMAT-0 medium, a two-interaction MACE foundation model [6] trained on the OMat24 data [7]. It has 128 channels, harmonics up to degree 3 (16 components), a 6 Å cutoff, and symmetric contractions up to third order. The second interaction receives 512 features per atom and has 10 paths, so each edge carries $10 \times 128 = 1{,}280$ radial weights. Its final readout contains a nonlinear layer.

Where a general implementation spends its work. The benchmark used throughout is $SrTiO_3$, $10 \times 10 \times 10$ cells, 5,000 atoms and about 370,000 directed edges (74 per atom in the perfect crystal). Storing the second interaction's weights alone takes $370{,}000 \times 1{,}280 \times 4$ bytes ≈ 1.9 GB per force call, and their gradient the same again. The standard MD loop adds a host-side integrator and a neighbour search at every step, with two transfers between host and device. Symmetrix-XL uses the same model, crystal, precision, cutoff and skin for its throughput and capacity measurements [2], which makes its published numbers a useful reference point.

3 Design and fidelity contract

FastMACE keeps MACE's own PyTorch modules and rewrites a private copy of the model. Engines such as Symmetrix-XL instead convert the checkpoint into their own representation [2]; Table 12 in Section 9 sets the two designs side by side. Figure 1 shows the structure.

Rewrites and their classes. Each change is a *rewrite* that the planner turns on for a model, precision and GPU. Every rewrite belongs to one of three classes, which the paper keeps apart:

- **Identity in real arithmetic**: composing linear maps, moving a linear map through a sum. Only rounding changes.
- **Rounding change**: a different summation order or a tensor-core decomposition of a product.
- **Sampled approximation**: radial tables and packed coefficients.

Some rewrites assume frozen parameters and provide no training gradients.

Software. The fast path is a PyTorch computation with specialised operators. PyTorch's compiler [8] fuses elementwise regions; Triton [9], run-time compiled CUDA, CUTLASS [10] and cuEquivariance [11] implement individual operations; automatic differentiation connects custom reverse kernels to the rest. Appendix C lists versions and the studies that deviate from them.

Supported domain. The implementation enforces several limits, which are separate from the gaps in validation. Every path requires periodic boundaries in all three directions. The tiled path of Section 6 accepts two-interaction MACE models of the standard scale-shift form only; it rejects models with joint embeddings and the polarisable family. Its per-tile neighbour lists need a cell at least three cutoff lengths wide along every lattice direction. The radial tables cover distances from 0.3 Å (Section 5.1).

3.1 The acceptance rule

A rewrite that changes rounding or approximates is adopted only if it passes the following rule. Production results are compared with MACE evaluated in float64. The yardstick is the reference implementation in float32: MACE under ASE [12], with forces and stresses from its float32 automatic differentiation, and with the energy summed in float64 from float32 site energies and the checkpoint's float64 atomic energies. For energy per atom, the largest force component and the largest stress component, each over the test configurations,

$$e_{\text{production}} \;\leq\; \max\left(3\, e_{\text{float32}},\; f\right), \tag{3}$$

with floors f of 0.01 meV/atom, 10^{-5} eV/Å and 10^{-7} eV/Å^3.

Test configurations. There are 18: for each of $SrTiO_3$ (135 atoms), NaCl (216) and $NbCl_5$ (1,944), all $3 \times 3 \times 3$ supercells, three cells with a random isotropic strain and three with a random symmetric strain tensor, each within $\pm 2\,\%$, followed by a Gaussian displacement of every coordinate with a standard deviation of 0.05 Å.

What the rule is and is not. It is a sampled test of static quantities, run when a rewrite is adopted. It is not run at every setup: an optional check applies it to the user's structure, and that check is off by default. It does not test energy conservation, relaxation behaviour or trajectories (Section 7). The factor was 2 until 7 November 2026; Appendix A gives the reason for the change.

3.2 How defaults are chosen

A rewrite becomes a default for a GPU family only after a paired timing in one process shows it faster in both build orders and the acceptance rule passes. The timing alternates blocks of 256 MD steps between the two sides, six blocks each after 200 warm-up steps, and reports for each side the median of its six block means; the ratio of the two medians is the result (Section 8.1). The noise floor of this comparison was measured at 0.2 % to 1.1 % depending on the machine. A rewrite that only removes work is enabled everywhere after one such test. Defaults therefore differ between GPUs (Table 2).

4 Reassociation with frozen parameters

4.1 Aggregate before projecting in the first interaction

In the first interaction the sender features depend only on the species. Together with the fact that the last layer of the radial network is linear, this allows the projection onto the 128 channels to move outside the neighbour sum. With $a_k(r)$ the hidden radial features (64 per species pair, cutoff included),

$$S_{i,z,b,k} = \sum_{j \in \mathcal{N}(i),\ z_j = z} a_k(r_{ij}; z_i, z)\ Y_b(\hat{r}_{ij}), \tag{4}$$

$$m_{i,b,c} = \sum_{z,k} W^{(b)}_{z,c,k}\ S_{i,z,b,k}, \tag{5}$$

where W combines the frozen radial matrix, the species features and the path normalisation. This is the density trick of the atomic cluster expansion [3] applied to MACE's first layer. Later interactions have environment-dependent sender features and cannot use it; there, Symmetrix-XL applies a learned linear map once per receiver after the neighbour sum [2], which is a related but different move.

Cost. Let n be the neighbours per atom, $K = 64$ the radial features, $C = 128$ the channels, $L = 16$ the angular components, $P = 4$ the paths of this interaction and S the number of species in the structure. Without the rewrite every edge forms $PC = 512$ weights from K features, which is $KPC = 32{,}768$ multiply-adds, and a message of $LC = 2{,}048$ numbers. With it, every edge adds $LK = 1{,}024$ numbers to one block, and every atom pays $SLKC = 131{,}072\,S$ multiply-adds for the projection. Per atom the rewrite therefore replaces about $35{,}000\,n$ operations by $1{,}000\,n + 131{,}000\,S$. It pays once n exceeds about four times S; for $SrTiO_3$, $n = 74$ and $S = 3$. The price is storage: SLK floats per atom, or $4.1\,S$ kB, 61 MB for the benchmark. S counts the species of the structure at hand, not the 89 elements of the model. These are operation counts from the formulas, not measurements, and the unfavourable case of many species with few neighbours has not been timed.

The implementation evaluates the radial table inside the edge walk and never writes an edge-by-feature array. Its reverse recomputes values and slopes, contracts the local adjoint into a distance derivative at once, and discards it. In the profile of Table 1 the first interaction's forward and reverse kernels take 323 μs of a 3,258 μs force call on the H200. In our profile of Symmetrix-XL 0.1.2 on the same GPU the first interaction took 6.4 ms (Figure 4), which is where most of the difference between the two engines arose on that date.

4.2 Composing adjacent linear maps

With frozen parameters, adjacent linear maps can be multiplied during preparation. FastMACE folds the linear map after the first product into its three consumers (the second interaction's input projection, its species-dependent skip map and the first readout), composes the first interaction's projection with its linear map and species skip into one matrix per receiver species and angular degree, and composes the map into the final readout with the preceding product, so that the readout tail runs at 16 instead of 128 channels. Nonlinearities stay in place.

4.3 A generated symmetric contraction

The first symmetric contraction maps the 16 angular components of a channel to one scalar and one vector, as a polynomial of degree up to three with 29 species-dependent weights per channel. For a given checkpoint it is a fixed polynomial.

Generation. The source is cuEquivariance's description of the contraction as a list of 2,039 coupling paths. Paths with the same weight, inputs and output are merged, which leaves 480 terms over 410 monomials. The generator emits each distinct product of two inputs once and extends it to the cubic monomials that contain it. For example, with $p = x_0 x_0$,

$$t_1 = \tfrac{1}{2}\,p, \qquad t_2 = c\,(p\,x_0), \qquad t_3 = \tfrac{1}{2}\,(p\,x_1), \qquad t_4 = \tfrac{1}{2}\,(p\,x_2), \qquad \dots \tag{6}$$

so x_0^2 is formed once for four terms. The result is a straight-line program of 478 multiplications and 541 fused multiply-adds per channel; the library kernel rebuilds each monomial and uses 901 multiplications. The reverse is the reverse mode of the same program. The division by the neighbour density that follows the first interaction is fused into the kernel.

A factored basis. The 29 weighted polynomials span 13 scalar invariants and 16 vector covariants. A second variant replaces them by a basis that is cheaper to evaluate, built from shared pair products, with the change of basis folded into the species weights once at setup. Its first elements are the squared norm of the l=1 part and its product with the scalar,

$$v_5 = \tfrac{1}{\sqrt{3}}\,(x_1^2 + x_2^2 + x_3^2), \qquad v_6 = x_0\,v_5. \tag{7}$$

The basis is chosen greedily by cost with a rank test, and the change of basis is fitted by least squares in float64. If the residual exceeds 10^{-5} the factored variant is refused and the first one is used. It needs 413 operations and 61 output multiply-adds per channel where the first variant needs about 1,040 in the forward pass.

Checks and effect. A test compares both variants with the library kernel on the output and on both gradients to a relative 10^{-5}; no numeric comparison is run at setup beyond the residual test above. On the H200 the four kernels this replaces took 179.5 μs per force call and the generated pair takes 82.4 μs.

Two kinds of generated code. This contraction is *synthesised*: its program is derived from the checkpoint's polynomial, with the sharing and the change of basis described above. The streaming kernels of Section 5.2 are also specific to the model, in a simpler way: their CUDA source is rendered from templates with the model's coupling paths, tensor layout and spline grid, and compiled at run time. Generating kernels per checkpoint is therefore common to FastMACE and Symmetrix-XL, which does so as a matter of design [2]. The difference that remains is where the kernels sit: in FastMACE they are operators inside PyTorch's execution of the model and its automatic differentiation.

5 Streamed execution

5.1 Radial functions as tables

A frozen radial network is a function of one variable per species pair. FastMACE evaluates it in float64 at setup and stores a piecewise polynomial, which removes the radial embedding, the hidden layers and the cutoff from the force loop. Tabulating an embedding network is established practice for Deep Potential models [13], and Symmetrix-XL projects MACE's radial networks onto cubic Hermite splines [2]. The forms used here follow from what each interaction consumes.

First interaction. It needs the hidden features $a_k(r)$. They are stored as Chebyshev series [14] of degree 5, together with the cutoff and density terms. The pieces start as eight uniform intervals and are halved until values and derivatives at test points agree with the network to 10^{-7} of their scale. For the nine species pairs of $SrTiO_3$ this gives 619 pieces and a 15 MB table.

Second interaction. It reads the 1,280 weights directly from cubic Hermite splines, evaluated in the power basis,

$$w(r) \approx c_0 + t\,(c_1 + t\,(c_2 + t\,c_3)), \qquad t = (r - r_k)/h_k. \tag{8}$$

This removes the hidden-to-weight matrix product from every force call. The knots are uniform within bands and denser at short distances; the recorded tables have 731 to 736 intervals between 0.3 and 6 Å, about three quarters of them below 2.5 Å. Reading the weights from splines in a receiver-major kernel follows Symmetrix-XL, which uses 256 nodes for this model [2]. The banded knots and the test below are ours.

The setup test. A Chebyshev table or a spline is used only if it passes the following test at setup. For each species pair, 6,000 uniform and 6,000 random distances are drawn. On these, the float32 table and the float32 network are both compared with the float64 network, for values and for first derivatives, each normalised by the largest value of its output column. The table's largest error must not exceed the network's own, over the whole range and over distances from 1 Å separately. The recorded worst ratios are 0.36 for the Chebyshev table and 0.49 for the spline (its slopes; 0.01 for its values). Two properties limit what this shows. It is a sampled test, not a bound for every distance. And for the spline the same 12,000 points drive the refinement of the knots and give the final ratio, so the ratio is an acceptance criterion and not an independent estimate; the Chebyshev pieces are chosen on other points than those of its test. Symmetrix-XL gates its splines in a comparable way, on node values, interpolation error, derivatives and end-to-end energies, forces and stresses [2].

Domain. The tables cover distances from 0.3 Å to the cutoff. Below 0.3 Å the value is held at its value at 0.3 Å and the radial derivative is set to zero, without a warning. Geometries with shorter distances are outside the tested domain.

A packed table for the forward pass, outside the setup test. On the forward pass the table is the dominant stream of bytes. A packed form stores eight bytes per weight instead of sixteen: the cubic is replaced by a quadratic in a centred coordinate, with three fixed-point coefficients of 22, 20 and 16 bits and a shared six-bit exponent. It is the default on all four GPUs of Table 2. *The setup test above does not cover it.* The packed table is derived from

the tested cubic at setup without an error check; the only fallback is to the cubic when an exponent does not fit. Its qualification is offline and narrower: a test of the kind above for the three species of $SrTiO_3$ gave a value ratio of 0.16, and the acceptance rule passed on the three test crystals when the packed table was adopted. For another checkpoint or other species the packed values are unchecked.

The reverse is no longer consistent with the evaluated energy. The slopes of the packed quadratic would fail the test by three orders of magnitude, so the reverse kernel reads the float32 cubic, for the weight values in the adjoints of the features and the harmonics as well as for the slopes in the distance derivative. The adjoint it receives, however, comes from a forward pass with the packed weights. Schematically, let $m_p(R)$ and $m_c(R)$ be this interaction's messages with packed and with cubic weights, J_p and J_c their Jacobians, and $\phi(R, m)$ the rest of the model, which also depends on the positions directly: through the first interaction's readout, the skip features, the density and the pair energy. The returned force is

$$F \;=\; -\,\partial_R \phi(R, m_p) \;-\; J_c(R)^{\top}\, \partial_m \phi(R, m_p), \tag{9}$$

whereas the gradient of the reported energy has J_p in place of J_c, and that of an all-cubic evaluation has m_c in place of m_p. *The forces therefore match, in general, the energy gradient of neither evaluator*, and reporting the cubic energy instead would not change that. Whether they derive from some other potential is not established. They remain within the acceptance rule, but no energy-conservation run exists for this configuration (Section 7). Symmetrix-XL does not have this problem: its forces are the derivative of the same spline that gives its energy.

5.2 Streaming the second interaction

Forward. Work is assigned to receiver-and-channel pairs. One thread walks a receiver's edges, evaluates the radial weights, contracts sender features with the harmonics, and accumulates the 40 output components of its channel in registers. The result is written once, already in the block layout of the following linear maps. Neither the edge messages nor the radial weights are materialised. Edges are staged in tiles of 32 in shared memory, and each receiver's edges are sorted by length so that neighbouring receivers read the same table pieces.

The kernels descend from the receiver-streaming convolution of Fuchs [15] in OpenEquivariance [16], which adapts the schedule of Sobek [17]; the adapted files keep the upstream licence. As there, the source of the forward and reverse kernels is rendered at setup from the model's coupling paths, tensor layout and spline grid and compiled at run time. The receiver-major layout with spline weights was taken from Symmetrix-XL after a profile on 6 November showed its forward at 952 μs on the H200 against 1,713 μs for ours, 1,623 μs in the kernel and 90 μs in the radial table that fed it; with that layout ours takes 629 μs (Table 1).

Reverse without stored weights. For one edge, path and channel let g_o be the adjoint of the receiver's message. With

$$u_a = \sum_{o,b} C_{oab}\, g_o\, Y_b, \qquad v_b = \sum_{o,a} C_{oab}\, g_o\, x_a, \qquad q = \sum_{a} x_a u_a, \tag{10}$$

the three adjoints are

$$\bar{x}_a \mathrel{+}= w\, u_a, \qquad \bar{Y}_b \mathrel{+}= w\, v_b, \qquad \bar{r} \mathrel{+}= w'\, q. \tag{11}$$

Algorithm 1 gives the kernel. It rebuilds w and its slope w' from the coefficients of Eq. (8), so the edge-weight gradient, a tensor of 1.9 GB in the general implementation, and the dense

Algorithm 1 Reverse pass of the second interaction for one edge $e = (j \to i)$ and channel c. The adjoint g of the receiver's message is read from a row staged in shared memory.

find the table interval k of r_e; $\quad t \leftarrow (r_e - r_k)/h_k$; $\quad \bar{r} \leftarrow 0$
for each path p **do**
  $(c_0, c_1, c_2, c_3) \leftarrow \mathrm{table}[k, p, c]$
  $w \leftarrow c_0 + t\,(c_1 + t\,(c_2 + t\,c_3))$ ▷ value
  $w' \leftarrow (c_1 + t\,(2c_2 + 3t\,c_3))/h_k$ ▷ slope from the same coefficients
  $u_a \leftarrow \sum_{o,b} C^{(p)}_{oab}\, g_{i,o,c}\, Y_b(\hat{r}_e)$
  $v_b \leftarrow \sum_{o,a} C^{(p)}_{oab}\, g_{i,o,c}\, x_{j,a,c}$
  $\bar{x}_{j,a,c}$ += $w\, u_a$ ▷ atomic add into the sender's row
  $\bar{Y}_{e,b}$ += $w\, v_b$
  $\bar{r}$ += $w' \sum_a x_{j,a,c}\, u_a$
end for
add $\bar{r}$ to the distance adjoint of edge e

	H200		A100 80GB PCIe	
Component	μs	share	μs	share
Second interaction, reverse	1,089	33 %	2,256	35 %
Second interaction, forward	629	19 %	1,255	19 %
Dense products (tensor cores)	609	19 %	1,069	16 %
First interaction, forward and reverse	323	10 %	844	13 %
Symmetric contractions	192	6 %	402	6 %
Other (geometry, elementwise, reductions)	416	13 %	686	11 %
Total	3,258		6,512	

Table 1: Kernel time of one force call (OMAT-0 medium, $SrTiO_3$, 5,000 atoms), from a hardware-counter profile of 9 November 2026 (revisions `c520e074` and `0df22417`). The A100 column is a PCIe board, not the SXM4 board of Table 4, whose own profile totals 6,209 μs in another configuration. The profiles predate the generated contraction on the H200 and the hybrid product on the A100.

projection through it never exist. The receiver's adjoint row is copied to shared memory once per run of edges.

Not storing per-edge weights in the reverse pass is not new: Symmetrix-XL's reverse pass also recomputes its radial and angular factors [2]. What is specific here is the setting. The kernel runs as one node of PyTorch's automatic differentiation, so the rest of the model keeps its ordinary reverse pass. Its sums for $\bar{x}$ and $\bar{Y}$ are those of the vendored OpenEquivariance kernel; our change replaces that kernel's store of the weight gradient by the product with w' in registers.

What limits these kernels. Table 1 shows that the second interaction is about 53 % of the kernel time of a force call. On the A100 (the PCIe board of the table) both of its kernels are bound by the fused multiply-add units (84–85 % busy); on the H200 they are bound by instruction issue. This shapes what further changes can gain (Section 10).

5.3 Dense products on tensor cores

After the edge work, dense per-atom products are the next largest cost. Tensor cores multiply at reduced precision, so a float32 operand is split into a high part and a residual [18], and three reduced-precision products recover the high–high term and the two cross terms [19]. The residual–residual term is dropped. FastMACE uses the CUTLASS kernel for this [10], with

GPU	Dense products	First symmetric contraction
H200	CUTLASS, three TF32 products	generated
A100-SXM4-80GB	CUTLASS, hybrid TF32 / FP16	generated, factored
RTX 4000 Ada	Triton, three TF32 products	generated, factored
RTX 5060 Ti	float32	cuEquivariance

Table 2: Routes chosen by measurement for OMAT-0 medium in float32. The differences reflect which unit limits each GPU. The packed forward table of Section 5.1 is on for all four.

the division by the density and the species selection fused into its output stage. The same three-product method has been deployed for another family of potentials [20]; what is specific here is its integration into MACE's linear maps and the hybrid variant below.

A hybrid product. On the A100 the two cross terms share one half-precision product. Let A_h be A with the low 13 bits of its significand cleared, which is the part TF32 keeps, and let the residual be scaled so that it fits half precision:

$$A'_l = 2^{11}(A - A_h), \qquad B'_l = 2^{11}(B - B_h), \tag{12}$$

both exact in float32. Then

$$AB \;\approx\; A_h B_h \;+\; 2^{-11}\,[\,A_h \;\; A'_l\,]\begin{bmatrix} B'_l \\ B_h \end{bmatrix}. \tag{13}$$

For every eight columns of the inner dimension the kernel issues one TF32 instruction (`m16n8k8`) for $A_h B_h$ and one FP16 instruction (`m16n8k16`) whose sixteen slots hold the pairs (A_h, A'_l) against (B'_l, B_h), each rounded to half precision. That dot product is 2^{11} times the sum of the two cross terms; it is scaled back and added with a fused multiply-add. All accumulation is in float32. The three-product kernel needs three TF32 instructions for the same step.

Since $|A'_l| < 2|A|$ must be representable in half precision, operands must stay below 32,752 in magnitude, and below 2^{-14} the cross term loses bits. There is no run-time guard, and the margin of the production graph is not established. An offline probe found the largest operand at 779, a factor 42 below the limit, but it ran the reference MACE model, whose intermediates are not the aggregated features, composed matrices and adjoints that the rewritten graph multiplies.

The accuracy was measured on synthetic operands, Gaussian matrices with log-normal row scaling in nine representative shapes, as

$$\max_{ij} \frac{|\widehat{C} - C|_{ij}}{(|A|\,|B|)_{ij}} \tag{14}$$

against a float64 product C. This bounds the error by the product of magnitudes; where terms cancel, the relative error of an output element can be much larger. The hybrid gave 1.1 to 2.7×10^{-7}, the same range as the three-product kernel. It ran at 0.85 to 1.00 of that kernel's time per shape and 0.87 in the sum.

5.4 Executing complete MD steps

Positions, velocities and forces stay on the GPU. A step replays two captured CUDA graphs [21]: the first integrates, checks displacements and compacts the neighbour candidates to the cutoff; the second evaluates forces and completes the integration (Figure 2). Graphs are captured

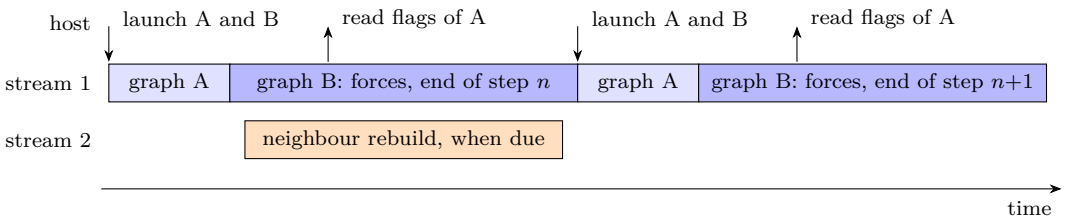


Figure 2: One MD step as two captured graphs (schematic, not to scale). Graph A integrates, checks displacements and compacts the neighbour candidates; graph B evaluates forces and completes the integration. The host launches both before it reads the flags of A, and a rebuild of the neighbour list runs on a second stream beside the force evaluation.

for padded edge counts, and padding contributes exactly zero. The engine provides Langevin, velocity-Verlet, stochastic-rescaling, Nosé–Hoover-chain and barostat integrators behind classes that follow ASE's interface [12].

Pipelining. The engine launches the force graph before the host has read the flags of the first graph. If the edge count has outgrown the capacity, or the neighbour list has expired, the engine restores the saved mid-step state and repeats the force stage with the same random numbers. The profiled runs had no repeated steps. The measured gaps between kernels were 63 μs in a 3.24 ms force graph on the H200.

Neighbour list. The list holds pairs within the cutoff plus a skin [22] and is rebuilt on the GPU. In fixed-cell runs the rebuild starts early, on a second stream, beside the force evaluation. For more than 10,000 atoms a cell-list screen replaces the pairwise screen of tiles (Section 8.6). Langevin noise is drawn in batches of 256 steps on a background thread.

None of this changes the time step, skips a force evaluation or reuses stale features.

6 Reduced-state tiled evaluation

The whole-system path keeps every intermediate of a force call in device memory. For larger systems FastMACE partitions the atoms into receiver tiles: blocks of a grid over the cell, capped at 8,192 or 16,384 receivers and ordered along one axis. A tile owns the edges that end at its receivers and gathers the sender halo they need. Evaluating a two-interaction model tile by tile in three sweeps, with only the inter-layer state global, is the tiled mode of Symmetrix-XL [2], which we follow. The following parts are our own.

Storing what the next interaction reads. Instead of the features h after the first product, FastMACE stores $x = Ah$ and $s = B_z h$, the two quantities the second interaction reads: 512 and 128 float32 values per atom, 2,560 bytes. The three sweeps are then:

1. evaluate the first interaction without a tape and store x and s;
2. evaluate the second interaction and differentiate, giving direct edge derivatives and the adjoints g^x, g^s;
3. recompute the first interaction and differentiate the surrogate $\Phi_T = \sum_{i \in T} \big[\varepsilon_i^{(1)} + \langle g_i^x, x_i \rangle + \langle g_i^s, s_i \rangle\big]$ with the adjoints held fixed.

Algorithm 2 Wavefront schedule over tiles $1, \ldots, T$ in groups of $g = 16$ consecutive tiles.

1: **for** each group G of g consecutive tiles **do**
2: **for** $t \in G$ and $u \in \text{needs}(t)$ whose sweep 1 has not run **do**
3: sweep 1 on u: take a row for each of its receivers and store x and s there
4: **end for**
5: **for** $t \in G$ **do**
6: sweep 2 on t: read x of its atoms and s of its receivers
7: add g^x to the accumulator (a row opens at an atom's first reader)
8: write g^s over s of its receivers
9: close every atom a with $\text{last}(a) = t$: copy its completed g^x from the accumulator over its x and free the accumulator row
10: **end for**
11: **for** each tile u with $\text{ready}(u) \leq \max G$ whose sweep 3 has not run **do**
12: sweep 3 on u: read g^x and g^s of its receivers, differentiate, return their rows
13: **end for**
14: **end for**

6.1 The wavefront schedule

In the plain schedule each sweep finishes before the next begins, so x, s and their adjoints are held for every atom. The wavefront schedule interleaves the sweeps so that an atom's row exists only between its first and its third sweep (Figure 3, Algorithm 2).

Dependencies. They are taken from the actual tile graphs, not from an assumed halo width. A first pass builds every tile's graph once and records three things: for a tile t, the set $\text{needs}(t)$ of tiles that own an atom of its graph, receivers and halo; for an atom a, the last tile $\text{last}(a)$ whose graph contains it; and for a tile u, $\text{ready}(u) = \max_a \text{last}(a)$ over its receivers. This pass costs 2 to 3.5 % of a call.

Why it is correct. Sweep 2 of a tile reads x for all atoms of its graph; they are owned by tiles in $\text{needs}(t)$, whose sweep 1 has run (line 2). A receiver's s is read only by its own tile's sweep 2, and afterwards only g^s is read, so g^s may overwrite s in place. An atom's x is read, and its g^x added to, only by the sweep 2 of tiles whose graph contains it. Between the first and the last of these tiles the original x and the partial g^x are both needed, so the partial sum lives in a separate accumulator; after $\text{last}(a)$ the sum is complete, x is no longer needed, and the completed g^x takes its row. Sweep 3 of a tile needs complete adjoints for its receivers, which holds once $\text{ready}(u)$ has run (line 11). Every tile therefore computes what it computes in the plain schedule; only the order of the additions into the forces changes.

Periodic boundaries. A halo atom across the boundary is owned by a tile at the other end of the sweep axis, so it enters needs and ready like any other. The consequence is that the first and the last layer are alive for the whole call. Both are split into a skin as wide as the cutoff plus the list skin, so that only the skins stay. The live band is then about five layers: three around sweep 2 and the two boundary skins.

What is bounded and what still grows. The workspace of a tile depends on the tile caps only, about 0.5 MB per receiver. Four things still grow with the atom count. The state rows: 2,560 bytes per atom that holds a row, and the share of atoms holding one at the same time falls with size, from 74–91 % at one million atoms to 30 % at ten million and 12 % at 149 million.

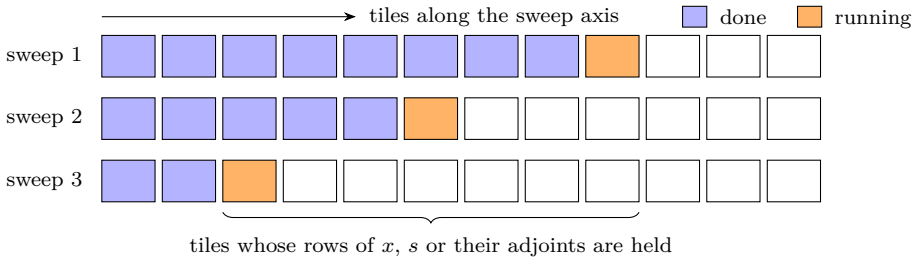


Figure 3: Wavefront schedule (schematic; the boundary skins are not drawn). The sweeps follow each other along one axis, so state is held only for the tiles between the last and the first sweep.

The accumulator for partial g^x: 2,048 bytes per atom between its first and its last reader. In float32 these two arrays take

$$M_{\text{state}} \;=\; 2{,}560\,N_{\text{live}} \;+\; 2{,}048\,N_{\text{open}} \quad \text{bytes}, \tag{15}$$

with N_{live} and N_{open} the peak numbers of atoms that hold a row and that have an open accumulator. Both pools are allocated at their peak size for the whole call, so the memory does not fall when the occupancy does. In a census at ten million atoms the rows took 770 bytes and the accumulator 92 bytes per atom of the system. Per-atom arrays on the device for forces, energies and index maps, 60 to 100 bytes per atom. And the neighbour lists kept between calls, about 3 bytes per edge on the host. Reusing the rows for the adjoints removes the separate arrays for g^x and g^s of the plain schedule: at 975,560 atoms the rows take 2.5 GB where separate arrays take 5.0 GB, to which the accumulator has to be added. The state can live on the device or in host memory. With host state, copies cost about 5 % of a call at the largest sizes; at 975,560 atoms on an RTX 5060 Ti they took 1.0 s of a 7.2 s call. Symmetrix-XL keeps its inter-layer state on the device and reaches larger systems by spreading them over GPUs; FastMACE has no multi-GPU path and reaches them by moving the state to the host.

Precision of the stored state. With host state and in-place rows, the default setting chooses the storage precision itself. It keeps float32 while the state fits in host memory, then drops the kept neighbour lists, and if the state still does not fit it stores float16. With float16 state the force errors are 12 to 51 times the float32 reference error, which is 4 to 17 times the limit, so it fails the acceptance rule. A caller who needs float32 throughout has to switch this choice off.

6.2 Coordinates

In boxes larger than about 64 Å float32 positions lose enough precision to produce visible force errors on a perfect crystal: 1.6×10^{-4} eV/Å at 975,560 atoms and 1.3×10^{-3} eV/Å at 149 million atoms and beyond. Since 11 November 2026 edge vectors are formed from float64 positions in a tile-local frame once the structure's extent reaches 64 Å, which reduces the residual to 3–4 $\times 10^{-6}$ eV/Å. The corrected path has been run up to 975,560 atoms, where it cost 2.8 to 2.9 % more than the float32 path on the A100 in a paired comparison. *None of the runs in Table 10 contains it*, including the two dated 11 November, whose revisions predate the fix: their own records show residuals of 1.7×10^{-4} eV/Å at 975,560 atoms and 1.3×10^{-3} eV/Å for the largest runs.

		Energy (10^{-3} meV/atom)		Forces (10^{-5} eV/Å)		Stress (10^{-7} eV/Å^3)	
GPU and date	Structure	prod.	ref.	prod.	ref.	prod.	ref.
H200, 9 Nov	$SrTiO_3$	1.79	0.37	2.71	3.19	1.25	3.02
	NaCl	0.09	0.14	0.44	0.48	0.39	0.50
	$NbCl_5$	0.40	0.23	7.90	8.23	0.70	0.58
A100-SXM4, 11 Nov	$SrTiO_3$	2.13	0.22	2.76	2.95	1.40	2.57
	NaCl	0.14	0.13	0.41	0.47	0.41	0.43
	$NbCl_5$	0.64	0.11	8.09	8.07	0.84	0.67

Table 3: Largest errors against the float64 reference over the six configurations of each structure, for the production path ("prod.") and for the float32 reference implementation ("ref."). The A100 rows use the hybrid tensor-core product. The floors of Eq. (3) are 10, 1 and 1 in the units of the three column pairs.

7 Numerical validation

Static errors. Table 3 gives the recorded errors under the rule of Section 3.1. In forces the production path is at or below the float32 error of the reference (ratios 0.85 to 1.00). In stress it is between 0.41 and 1.25 times that error; the A100 with three TF32 products instead of the hybrid reaches 1.31. The energy error is up to ten times that of the float32 reference (2.1×10^{-3} against 0.2×10^{-3} meV/atom on $SrTiO_3$) and passes through the absolute floor of 0.01 meV/atom, not through the factor of three. In absolute terms it is small.

Against the published numbers of Symmetrix-XL. For the same model and crystal in float32, Symmetrix-XL reports differences from the reference implementation of 1.2×10^{-5} eV/atom in energy (0.012 meV/atom), 6.5×10^{-5} eV/Å in forces and 8.8×10^{-8} eV/Å^3 in stress [2]. These are not directly comparable with Table 3: they are differences from the reference on that paper's configurations, ours are errors against float64 on small rattled cells. The benchmark that first compared the two engines on one machine (3 November) measured force differences from MACE with cuEquivariance in float32 of 0.8×10^{-5} eV/Å for FastMACE and 1.7×10^{-5} eV/Å for Symmetrix-XL.

Energy conservation: no record for today's defaults. The one whole-system record is an NVE run of NaCl (216 atoms, 10 ps) from 2 November: the drift per atom was -4×10^{-6} meV/ps with the Chebyshev tables and 9×10^{-7} meV/ps without them. It predates the spline weights and the packed forward table. The configuration that produced the step times of Section 8 has therefore never been run for energy conservation, although it is the one in which the reverse pass is not consistent with the evaluated energy (Section 5.1). Symmetrix-XL reports -9×10^{-7} meV/ps per atom over 100 ps on the 5,000-atom benchmark [2]. For the tiled path the only record is a 1 ps run of 40,000 atoms at 600 K (0.002 meV/ps per atom), too short to separate drift from fluctuation.

What is not validated.

- **Dynamics of the default configuration.** Missing are an identical configuration with the same cubic in forward and reverse, with its speed, and a comparison with the packed route by directional energy derivatives, by checks at the knots and by equilibrated NVE runs.
- **The packed table for other checkpoints and species** (Section 5.1).

- **The largest tiled runs**, which used the earlier coordinate path (Section 6.2).
- **Downstream properties.** No relaxation, phonon, transport or stability benchmark has been run. Symmetrix-XL reproduces the published Matbench Discovery results [23] and three further workflows; that qualification uses the MACE-MPA-0 checkpoint in float64, separately from its float32 timings and NVE run with OMAT-0.
- **Geometries with distances below 0.3 Å** (Section 5.1).

Passing the rule establishes numerical agreement with the reference on the 18 configurations. It is not a bound over all geometries and says nothing new about agreement with quantum-mechanical data.

8 Performance

8.1 Protocols and estimators

Unless stated otherwise: OMAT-0 medium, float32, $SrTiO_3$ with 5,000 atoms, Langevin dynamics at 300 K, 1 fs time step, 0.5 Å skin. Integration and neighbour maintenance are included; construction, compilation and warm-up are not. Three protocols appear.

- The **paired protocol** warms up for 200 steps and times six blocks of 256 steps per side, alternating between two configurations in one process. The reported time is the *median of the six block means*, and paired numbers are the two build orders. Where the mean over the whole window differs from the median, we say so.
- The **cold protocol** is that of an external benchmark and is used for the comparison with Symmetrix-XL on $SrTiO_3$. It starts from rest, discards 5 steps and times 20, in fresh processes: three for Symmetrix-XL and two or three for FastMACE. In 25 steps under the thermostat the system reaches 18 K, and *the timed window contains no neighbour rebuild*. It measures the force call and the integrator, not sustained MD at 300 K.
- The **thermal-start protocol** starts from a Maxwell–Boltzmann draw at 300 K and times 300 steps after 100, or 3,000 after 500. Neighbour rebuilds occur. Equilibration is not established: starting from the perfect lattice, the Symmetrix-XL run on $SrTiO_3$ moved from 187 to 233 K during its timed window, and FastMACE's temperatures were not logged.

Several adoption studies used other block counts or separate processes; Appendix C lists them.

8.2 Step times

Table 4 gives the recorded times. For $SrTiO_3$ on the H200 and the A100 the median and the mean over the window agree to 0.25 %. For $NbCl_5$ they do not, on either GPU: the first block is 60 % slower than the rest on the A100 and about 50 % slower on the H200, on both sides and in both orders, which puts the mean 10 % and 6 to 7 % above the median. The cause has not been examined. The median describes the steady state; a user's first 256 steps cost more. The H200 value for $NbCl_5$ also comes from another study and revision than the H200 value for $SrTiO_3$ (Appendix C). Block-level data for the RTX 4000 Ada and RTX 5060 Ti rows is not on record. Appendix A shows how the A100's time fell between 3 and 11 November.

GPU	Date of record	$SrTiO_3$, 5,000 atoms		$NbCl_5$, 1,944 atoms
		ms per step	μs per atom and step	ms per step
H200	9 Nov 2026	3.40 / 3.37	0.68	1.09[†]
A100-SXM4-80GB	11 Nov 2026	6.24 / 6.23	1.25	1.67[*]
RTX 4000 Ada	10–11 Nov 2026	17.67 / 17.76	3.5	3.88 / 3.90
RTX 5060 Ti	10–11 Nov 2026	$\approx$ 15.4	3.1	$\approx$ 3.42

Table 4: Recorded MD step times under the paired protocol: medians of six block means, two build orders where both are on record. Each row comes from its own adoption study and source revision; this is not a controlled comparison of the GPUs. [*]The mean over the window is 1.84 ms: the first block took 2.7 ms in both build orders and the other five 1.65 to 1.67 ms. [†]From the baseline side of a separate study at another revision, with eight blocks; the means over the window are 1.15 and 1.17 ms, the first block taking 1.6 ms.

Change	GPU	Date	Step time ratio (two orders)
Radial tables (first evidence)	A100 PCIe	3 Nov	0.902 / 0.901
Reverse rebuilds weights from the table	A100	4 Nov	0.943 / 0.943
Factored reverse	A100	4 Nov	0.938 / 0.940
Fused first-interaction forward and reverse	A100	5 Nov	0.966 / 0.962
Spline weights, receiver-major forward, composed maps (ten switches)	A100	8 Nov	0.747 / 0.747
Edges sorted by length	H200	8 Nov	0.981 / 0.979
Packed forward table	A100	9 Nov	0.956 / 0.955
	RTX 4000 Ada	11 Nov	0.904 / 0.903
Generated symmetric contraction	H200	9 Nov	0.976 / 0.973
	A100 PCIe	9 Nov	0.969 / 0.965
Shared geometry sums in the forward	A100	10 Nov	0.976 / 0.981
Hybrid TF32 / FP16 product	A100	11 Nov	0.990 / 0.988

Table 5: Paired step-time ratios (new over old, below 1 is faster; medians of block means) from the studies on which each change was adopted. The baselines differ from row to row, so the ratios do not multiply to a total. The first row used eight blocks of 200 steps; the note behind the rows of 4 and 5 November does not state their schedule.

8.3 Effect of individual changes

Table 5 lists the paired measurements behind the main changes, and Table 6 those of the tiled path. They are adoption records, not an ablation: each row has its own baseline and revision, one row bundles ten switches, and the first-interaction reassociation and the MD engine predate these studies and have no row. Being faster in both build orders shows a direction; it does not give an uncertainty, and for ratios within about 1 % of one, such as the hybrid product, the six blocks per side are the only evidence. Where the kernel saving and the step saving were both measured they agree: the generated contraction saved 97 μs of kernel time on the H200 and 84–93 μs per step. Memory was not recorded in these studies.

8.4 Comparison with Symmetrix-XL

Table 7 collects the runs in which both engines were measured on one machine in November. Two kinds of run have to be kept apart.

Change	State	RTX 4000 Ada	A100-SXM4-80GB
Nine speed switches	device	0.868 / 0.868	0.792 / 0.789
Nine speed switches	host, wavefront	0.772 / 0.769	0.782 / 0.785
Captured tile graphs	device	1.001 / 1.001	0.569 / 0.571

Table 6: Paired force-call ratios for the tiled path at 975,560 atoms of $SrTiO_3$ (10 and 11 November, two build orders).

				ms per step			GPU memory
GPU	Date	Run	Version, model, system	S-XL	FastMACE	Ratio	S-XL / FastMACE
A100 PCIe	3 Nov	cold	OMAT-0 medium, $SrTiO_3$ (5,000)	25.0	26.5	0.95	1.2 / 5.4 GB
H200	5 Nov	cold	0.1.1, same	14.86	5.66	2.63	–
H200	6 Nov	cold	0.1.2, same	14.57	5.72	2.54	–
H200	9 Nov	mixed	0.1.2, OMAT-0 small, $SrTiO_3$ (5,000)	11.48	2.12	(5.4)	–
RTX 5060 Ti	5 Nov	thermal	0.1.1, OMAT-0 medium, $SrTiO_3$ (5,000)	50.8	24.4	2.1	767 / 1,381 MiB
RTX 5060 Ti	5 Nov	thermal	0.1.1, OMAT-0 medium, NaCl (216)	3.58	1.15	3.1	455 / 720 MiB

Table 7: Same-machine Langevin runs of Symmetrix-XL ("S-XL", under LAMMPS) and FastMACE, each with the FastMACE defaults of its date. The ratio is the Symmetrix-XL time over the FastMACE time. "Cold" and "thermal" are the cold and the thermal-start protocols of Section 8.1: the cold runs contain no neighbour rebuild, the thermal-start runs 19 ($SrTiO_3$) and about 150 (NaCl) on the Symmetrix-XL side; FastMACE's rebuild counts and temperatures were not logged in these runs. The first row is from an external benchmark that does not state the version. In the H200 row of 5 November FastMACE ran with its A100 settings, which became its H200 defaults afterwards; with the generic settings it had until then the step took 13.1 ms (ratio 1.13). The OMAT-0 small row mixes protocols and is not a matched ratio: Symmetrix-XL ran cold, while the FastMACE time is from a separate run of 8–9 November with a thermal start, 200 steps after 100, and 13 to 14 rebuilds in the timed window.

Cold runs on the main benchmark. On 3 November FastMACE was 6 % slower on an A100; its radial tables were not yet enabled on that GPU. On the H200 two days later it was 2.6 times faster, and the ratio held for version 0.1.2. These ratios hold for the cold protocol: 25 steps from rest, 5 to 18 K, no rebuild. They say nothing about the overlapped neighbour rebuild of Section 5.4, and they are not a ratio for equilibrated MD at 300 K. A longer Symmetrix-XL run on the H200 (300 steps after 100, heating to 171 K, with rebuilds) took 15.7 ms per step, 6 % more than its cold time; no matching FastMACE run exists.

The smaller model. For OMAT-0 small the two times on record were taken under different protocols (Table 7), so their quotient of 5.4 is not a matched ratio and is not used further. What the records do show is that Symmetrix-XL gained a factor 1.27 from the smaller model, both of its runs being cold.

Runs with thermal initial velocities. The two runs on the RTX 5060 Ti start from thermal velocities and include rebuilds. There FastMACE was 2.1 and 3.1 times faster. They are closer to MD conditions than the cold runs but not equilibrated (Section 8.1), and they are on one

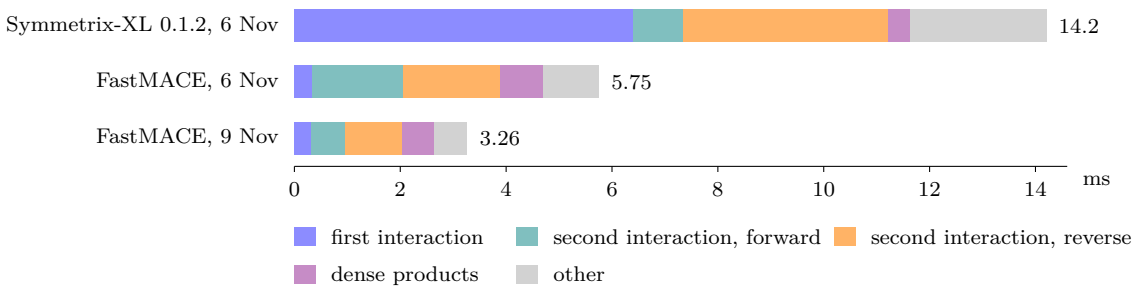


Figure 4: Kernel time of one force call on the H200 by stage ($SrTiO_3$, 5,000 atoms), from three separate profiles: Symmetrix-XL 0.1.2 under a tracing profiler on 6 November, FastMACE on the same day, and FastMACE on 9 November (Table 1). "Other" includes the symmetric contractions. In the FastMACE profile of 6 November a separate radial-table kernel (0.09 ms) fed the second interaction and is counted with its forward.

consumer GPU and one date.

Memory. FastMACE used more GPU memory in every run: 1.6 to 1.8 times on the RTX 5060 Ti, and 4.5 times in the A100 run of 3 November, which predates the streamed kernels of Section 5.2. Automatic differentiation keeps intermediates that a hand-scheduled reverse pass does not.

Where the difference arose. Figure 4 shows the stages of the force call in the configurations that were profiled. Geometry, edge counts and padded capacities change with the protocol, so these are the stage costs of those profiles, not of every run. On 6 November Symmetrix-XL spent 6.4 ms in the first interaction, which FastMACE had already reduced to 0.3 ms by the rewrite of Section 4.1, and 3.9 ms in the reverse of the second interaction against 1.8 ms. Its forward of the second interaction was the faster one, 0.95 against 1.71 ms with the radial table that fed ours, and FastMACE adopted that layout two days later.

Since then. FastMACE's step has fallen from 5.72 to 3.40 ms on the H200 and from 24.4 to about 15.4 ms on the RTX 5060 Ti. We have not rerun Symmetrix-XL since 9 November, and its later releases may be faster, so these numbers do not give a current ratio.

Its published numbers. For this model and crystal, the Symmetrix-XL paper reports 6.15 μs per atom and step on an A100-SXM4 80 GB at 1.25 million atoms, and 6.41 μs in tiled mode at 11.2 million atoms [2]. FastMACE's recorded values on the same GPU model are 1.25 μs per atom and step at 5,000 atoms and 2.1 μs per atom for a tiled force call at 0.98 million atoms (Section 8.6). The sizes, the step loop and the timing protocol differ, so this is an indication, not a measured ratio.

8.5 Comparison with general engines

Table 8 collects the recorded comparisons with engines that run MACE through its own modules. *All of them were made in October with earlier versions of FastMACE*, and the other engines have moved on as well. They are single-structure runs; TorchSim and the ALCHEMI Toolkit are built for batches of structures, which this comparison does not exercise. With these caveats, FastMACE was between 2.8 and 15 times faster per step on these small cells. We have no

Engine	Model	ms per step, NaCl / $NbCl_5$ other engine	FastMACE	Date
ASE 3.27 with MACE's calculator (cuEquivariance)	MH-1 [24]	44.1 / 135.2	2.99 / 21.5	24 Oct
TorchSim 0.6.2 [25], best compile mode	OMAT-0 medium	16.3 / 39.4	1.62 / 9.79	25 Oct
ALCHEMI Toolkit 0.2.0 [26]	OMAT-0 medium	15.8 / 27.3	1.65 / 9.91	26 Oct

Table 8: Recorded Langevin step times of general engines and of FastMACE on an RTX 5060 Ti, for NaCl (216 atoms) and $NbCl_5$ (1,944 atoms), each pair on the same day and machine. Every run is unbatched: one structure on the GPU. The three studies used thermal starts and their own step counts and software environments (Appendix C).

Atoms	ms per step	μs per atom and step	Date of record
5,000	15.4	3.1	10–11 Nov
20,480	64.9	3.2	11 Nov
40,000	127.8	3.2	11 Nov
69,120	222–224	3.2	11 Nov
214,375*	1,526	7.1	9–10 Nov

Table 9: MD step time of the whole-system path against system size on an RTX 5060 Ti (16 GB), $SrTiO_3$. The rows come from four studies with different timing schedules (Appendix C); the 69,120-atom values are from separate processes. *A different configuration from the rows above: memory-saving rewrites are active and the neighbour rebuild still uses the pairwise screen.

measurement against the LAMMPS interface with cuEquivariance [27], which is the baseline of the Symmetrix-XL paper.

8.6 Larger systems

Neighbour search. The pairwise screen of the rebuild grows with the square of the atom count. On an RTX 5060 Ti a rebuild takes 25 ms at 20,480 atoms and 2.1 s at 214,375 atoms; the cell-list screen brings these to 8 and 73 ms with a bit-identical list. In MD the step time ratio was 0.98 at 20,480 atoms and 0.96 at 40,000 atoms.

Cost against size. On the RTX 5060 Ti the step costs 3.1 to 3.2 μs per atom from 5,000 to 69,120 atoms (Table 9). At 214,375 atoms, the largest system that fits, it costs 7.1 μs. Two things differ in that last row and cannot be separated from the record: the planner had turned on rewrites that recompute instead of store, and the run predates the cell-list screen on this GPU, whose effect at that size we estimate at about 10 % of the step. The RTX 4000 Ada went from 3.5 μs per atom at 5,000 atoms to 6.9 at 296,595 under the same two conditions.

Capacity. Table 10 gives the largest completed runs. The whole-system path reaches 13,000 to 15,000 atoms per GB of device memory; the values Symmetrix-XL reports for its standard mode correspond to 15,000 to 16,000 [2]. The tiled path with the wavefront schedule and host state has completed force calls on 180 million atoms with a 20 GB card and on 320 million with an 80 GB card. In the first case the device was the limit and the host held 129 GB, or 0.7 kB per atom; in the second the host limit of 231 GB was the constraint.

Path	GPU	Date	Atoms ($SrTiO_3$)	Time
Whole system, MD	RTX 5060 Ti, 16 GB	9–10 Nov	214,375	1.53 s per step
Whole system, MD	RTX 4000 Ada, 20 GB	9–10 Nov	296,595	2.04 s per step
Tiled, device state	A100-SXM4-80GB	11 Nov	975,560	2.04 s (2.1 μs/atom)
Tiled, device state	RTX 5060 Ti	11 Nov	975,560	6.1 s (6.2 μs/atom)
Tiled, wavefront, host state	RTX 4000 Ada, 20 GB, 251 GiB host	10 Nov	179,685,000	1,773 s (9.9 μs/atom)
Tiled, wavefront, host state	A100-SXM4-80GB, 231 GB host limit	9–10 Nov	320,000,000	3,585 s (11.2 μs/atom)
Tiled, wavefront, host state in float16 (fails the rule)	A100-SXM4-80GB	9–10 Nov	397,535,000	3,913 s (9.8 μs/atom)

Table 10: Largest completed runs per path. The tiled rows are single force calls, not MD steps; 975,560 atoms is the largest device-state size on record, not a limit. Every row used the float32 coordinate path that Section 6.2 replaces: the source revision of each run, including the two dated 11 November, predates the fix.

Model	GPU	Structure (atoms)	ms per step	Date
OMAT-0 small	H200	$SrTiO_3$ (5,000)	2.12	9 Nov
OMAT-0 small	A100-SXM4	$SrTiO_3$ (5,000)	4.19	9 Nov
MACE-OFF23 S / M / L	A100-SXM4	water (648)	1.49 / 2.60 / 6.92	8 Nov
MACE-POLAR-1 S / M / L	A100-SXM4	NaCl (216)	2.29 / 4.23 / 7.71	8 Nov
MACE-POLAR-1 S / M / L	A100-SXM4	$NbCl_5$ (1,944)	7.77 / 16.38 / 29.06	8 Nov

Table 11: Recorded MD step times for other models with their own defaults: OMAT-0 small, the organic force fields MACE-OFF23 [28] in three sizes (S, M, L), and the polarisable MACE-POLAR-1 models [29], whose long-range part runs in reciprocal space. The record summaries do not state the total charge and spin used for MACE-POLAR-1. The timing schedules differ between the three groups (Appendix C).

These are *capacity demonstrations, not accuracy-qualified results.* They are single force calls, not MD. Like every row of the table they used the earlier coordinate path, so the forces of the largest runs carry the 1.3×10^{-3} eV/Å error of Section 6.2; whether the corrected path reaches the same sizes at the same cost has not been run. And the storage precision was left to the automatic choice of Section 6.1, which kept float32 at 180 and 320 million atoms and stored float16 at 398 million. Symmetrix-XL reports 11.2 million atoms on one A100 with all state on the device, and 703 million atoms in MD on 64 GPUs [2].

Capturing the compiled tile bodies in CUDA graphs reduced the A100's tiled force call at 975,560 atoms from 3.59 to 2.04 s (Table 6). The tiled path then costs 2.1 μs per atom against 1.25 for the whole-system path at 5,000 atoms.

8.7 Other models

The mechanisms are not specific to OMAT-0 medium, although the defaults differ by model. Table 11 lists recorded step times for other MACE families under their own defaults; each configuration passed the acceptance rule. No reference implementation was timed on the same machines, so the table shows that the path runs, not how much it gains. Nor does it test generality in the sense that matters for Section 4.1: all structures are ordered crystals or water with two to four species. A melt or a disordered alloy, and a series in the number of species, have not been run.

Aspect	Symmetrix-XL	FastMACE
Model	Checkpoint converted to its own format; native evaluator	MACE's PyTorch modules, rewritten in a private copy
Kernels	Generated per model, precision and target; cached	Streaming kernels rendered from templates for the model's paths, layout and spline grid; first contraction synthesised as a polynomial program; the rest general; routes chosen by a planner per GPU
Forces	Hand-scheduled reverse stages	Automatic differentiation with custom reverse kernels
Radial functions	Cubic Hermite splines, 256 nodes; forces from the same spline	Chebyshev tables and banded cubic splines; packed forward table by default
Dense products	float32	Tensor cores (three TF32 products, or TF32 with FP16)
MD loop	LAMMPS or ASE	Own GPU-resident engine in captured graphs
Large systems	Tiled, three sweeps, state on the device; MPI over GPUs	Tiled, three sweeps, reduced state, wavefront, optional host state; one GPU
Hardware	NVIDIA GPUs and CPUs; a code path for AMD GPUs	NVIDIA GPUs
Validation	Differences from the reference and 100 ps NVE (OMAT-0, float32); published downstream benchmarks (MACE-MPA-0, float64)	Numeric rule per rewrite on 18 small cells; 10 ps NVE on an earlier configuration
Code	Public	Private at the time of writing

Table 12: Design of Symmetrix-XL and FastMACE. The Symmetrix-XL column is taken from its paper [2], its documentation as read on 4 November 2026, and our profile of version 0.1.2.

9 Related work

Kernels for equivariant tensor products. e3nn [5] provides the general language of MACE's operations, and cuEquivariance [11] optimised implementations of them. OpenEquivariance [16] and FlashTP [30] generate tensor-product kernels that fuse the Clebsch–Gordan contraction with the aggregation over neighbours. FastMACE builds on cuEquivariance and OpenEquivariance. Sobek [17] introduced the streaming schedule for equivariant convolutions that both our second-interaction kernels and Symmetrix-XL's descend from. These libraries accelerate one operation and leave the surrounding model program to the framework.

Inference engines for MACE. Symmetrix [31] evaluates MACE models natively, and Symmetrix-XL [2] extends it with streamed edges, per-model generated kernels and tiling. Table 12 compares its design with ours. Each approach pays for what it gains. A native evaluator schedules its own reverse pass and stores no intermediates for automatic differentiation, which is why it uses less memory; it has its own back ends for CPUs and for AMD GPUs; and running under LAMMPS gives it that code's integrators and its domain decomposition over GPUs. Keeping MACE's modules means that every part FastMACE has not rewritten still runs through the reference code, that a rewrite can be switched off and compared in one process, and that new MACE variants run before they are optimised. TorchSim [25] and the ALCHEMI Toolkit [26] run MACE through its own modules with batched or compiled execution.

Deployment of other equivariant potentials. The NequIP and Allegro infrastructure compiles models and uses optimised tensor-product kernels [32, 33], and strictly local models have been scaled to very large systems [34]. chemtrain-deploy [35] and the parallel algorithm of Park et al. [36] run message-passing potentials with domain decomposition in LAMMPS [27, 37]. A different route to speed is a cheaper model, as in the neuroevolution potentials of GPUMD [38]; FastMACE and Symmetrix-XL instead keep the trained model and change its execution.

Tables and tensor cores. Tabulating embedding networks is used for Deep Potential models [13]. Splitting a float32 operand into a reduced-precision part and a residual, with four tensor-core products, goes back to Markidis et al. [18]. Ootomo and Yokota [19] refined it: they accumulate in float32 outside the tensor cores, scale the residual against underflow, use TF32, and drop the product of the two residuals, which leaves three products. The numerical behaviour of the hardware is analysed in [39]. NVIDIA has described the three-product method for the PET family of potentials through CUTLASS, and a variant with nine bfloat16 products on newer hardware [20]; it reports force errors of about three times the run-to-run float32 difference and energy conservation close to float32. Neither variant combines the two cross terms. Our hybrid product replaces two of the three passes by one half-precision pass.

10 Limitations and open experiments

- **No configuration supports all claims at once.** Step times, comparisons, accuracy and capacity come from different revisions and settings (Appendix C).
- **Dynamics of the default configuration are unvalidated**, and its packed table is checked offline for one set of species only (Sections 5.1 and 7).
- **The main comparison with Symmetrix-XL is a cold run without rebuilds** (Section 8.4).
- **The largest capacity runs predate the coordinate fix** and left the storage precision to an automatic choice (Section 8.6).
- **No ablation on one revision, and no uncertainty for small gains.** Memory was not measured alongside time.
- **Narrow workloads.** Timings are for ordered crystals with few species. The first-interaction rewrite and the table storage both depend on the number of species.
- **No downstream validation**, and the energy error passes through an absolute floor.
- **One GPU, NVIDIA only, one structure at a time.** There is no multi-GPU or CPU path. The largest systems are single force calls of half an hour to an hour, not MD.
- **Memory.** FastMACE used 1.6 to 1.8 times the GPU memory of Symmetrix-XL in the runs of 5 November.
- **No run-time guards** for the operand range of the hybrid product or for distances below 0.3 Å, and the operand range of the production graph has not been measured.
- **Start-up cost** is excluded from all timings. Recorded setup times for the benchmark were 33 to 48 s with warm caches and 94 to 110 s on a first run; in the one record that separates it, fitting the radial tables took about 35 s of a first build.
- **Remaining headroom.** On the A100 the two largest kernels are bound by one arithmetic unit, and paths around it were measured and closed: a half-precision path for distant

	Experiment	Resolves
1	Today's defaults against the same cubic in forward and reverse: speed, directional energy derivatives, checks at the knots, equilibrated NVE on the benchmark and on a hot or disordered structure	Whether the timed MD configuration is valid
2	FastMACE and Symmetrix-XL on one GPU, equilibrated, long enough for representative rebuilds; force-call time, step time, rebuild count and temperature reported separately	Sustained comparative throughput
3	The largest tiled runs again with the corrected coordinate path and float32 state fixed	Accuracy-qualified capacity
4	Ablations and the size curve on one revision, with timing distributions and memory	Attribution and reproducibility
5	Disordered, hot and composition-diverse workloads	Generality beyond the main crystal

Table 13: Open experiments.

edges failed the accuracy rule, and a double-precision path cost more registers than it saved time. Further gains at this model size are expected to be a few per cent each.

Table 13 lists the experiments that would close the first six items, in order of what they resolve.

11 Conclusion

FastMACE shows how far the evaluation of a fixed equivariant potential can be reorganised without leaving the framework it was trained in. Reassociation removes most of the first interaction's edge work, streamed kernels remove the per-edge weight tensors of the second, and a reduced inter-layer state with a wavefront schedule takes the tiled path to hundreds of millions of atoms on one GPU. For the model studied here the recorded step time on an A100 is 6.2 ms and on an H200 3.4 ms, with static force and stress errors at the level of the float32 reference. In a cold 25-step benchmark FastMACE was 2.5 to 2.6 times faster than Symmetrix-XL on the main benchmark, and 2.1 and 3.1 times in two runs with thermal initial velocities and neighbour rebuilds. Symmetrix-XL remains ahead in validation, in memory use and in reach across hardware and system size. What the paper does not yet have is one configuration whose speed, accuracy, dynamics and capacity are shown together; the experiments of Table 13 are aimed at that.

Acknowledgements

Part of the measurements were run on the Alex cluster of NHR@FAU.

Use of AI tools. Large language models were used throughout this project. The implementation and the experiments were carried out in LLM coding sessions directed by the author; several mechanisms were first proposed in LLM-written research reports (Appendix B); and this manuscript was drafted and revised with LLM assistance from the project's records, including an LLM-written review.

Code and data availability

The repository is private at the time of writing. Appendix C names the record behind each number, but a reader cannot check private directory names. Before submission the evidence needs a public repository or archive with the checkpoint and structure identities, the resolved configuration of each study, and the commands that produced it.

References

[1] Ilyes Batatia, Dávid Péter Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2206.07697.

[2] Lei Fu, Zihui Feng, Yongheng Li, Hongwei Du, Xin He, Junyi Wu, Kejie Bao, Yueyu Zhang, Zeyu Deng, Ziheng Lu, and Bonan Zhu. Train for accuracy, execute at scale: Architecture-preserving inference for equivariant atomistic foundation models. *arXiv preprint arXiv:2610.01036*, 2026.

[3] Ralf Drautz. Atomic cluster expansion for accurate and transferable interatomic potentials. *Physical Review B*, 99(1):014104, 2019. doi: 10.1103/PhysRevB.99.014104.

[4] Simon Batzner, Albert Musaelian, Lixin Sun, Mario Geiger, Jonathan P. Mailoa, Mordechai Kornbluth, Nicola Molinari, Tess E. Smidt, and Boris Kozinsky. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1):2453, 2022. doi: 10.1038/s41467-022-29939-5. arXiv:2101.03164.

[5] Mario Geiger and Tess Smidt. e3nn: Euclidean neural networks. *arXiv preprint arXiv:2207.09453*, 2022.

[6] Ilyes Batatia, Philipp Benner, Yuan Chiang, Alin M. Elena, Dávid P. Kovács, Janosh Riebesell, Xavier R. Advincula, Mark Asta, Matthew Avaylon, William J. Baldwin, et al. A foundation model for atomistic materials chemistry. *The Journal of Chemical Physics*, 163:184110, 2025. doi: 10.1063/5.0297006. arXiv:2401.00096.

[7] Luis Barroso-Luque, Muhammed Shuaibi, Xiang Fu, Brandon M. Wood, Misko Dzamba, Meng Gao, Ammar Rizvi, C. Lawrence Zitnick, and Zachary W. Ulissi. Open materials 2024 (OMat24) inorganic materials dataset and models. *arXiv preprint arXiv:2410.12771*, 2024.

[8] Jason Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 2024. doi: 10.1145/3620665.3640366.

[9] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019. doi: 10.1145/3315508.3329973.

[10] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Aditya Atluri, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS. Software, https://github.com/NVIDIA/cutlass, 2023. Version 4.8.1 used here.

[11] NVIDIA. cuEquivariance. Software, https://github.com/NVIDIA/cuEquivariance, 2026. Version 0.12.0 used here.

[12] Ask Hjorth Larsen, Jens Jørgen Mortensen, Jakob Blomqvist, Ivano E. Castelli, Rune Christensen, Marcin Dułak, Jesper Friis, Michael N. Groves, Bjørk Hammer, Cory Hargus, et al. The atomic simulation environment—a Python library for working with atoms. *Journal of Physics: Condensed Matter*, 29(27):273002, 2017. doi: 10.1088/1361-648X/aa680e.

[13] Denghui Lu, Wanrun Jiang, Yixiao Chen, Linfeng Zhang, Weile Jia, Han Wang, and Mohan Chen. DP compress: A model compression scheme for generating efficient deep potential models. *Journal of Chemical Theory and Computation*, 18(9):5559–5567, 2022. doi: 10.1021/acs.jctc.2c00102. arXiv:2107.02103.

[14] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2013.

[15] Paul Fuchs. Add receiver-streaming UVU convolution kernel. OpenEquivariance pull request 223, https://github.com/PASSIONLab/OpenEquivariance/pull/223, 2026.

[16] Vivek Bharadwaj, Austin Glover, Aydin Buluc, and James Demmel. An efficient sparse kernel generator for O(3)-equivariant deep networks. In *Proceedings of the SIAM Conference on Applied and Computational Discrete Algorithms (ACDA25)*, 2025. arXiv:2501.13986.

[17] Vladimir Chorošajev and Cédric Bény. Sobek: Streaming equivariant tensor product convolutions. *arXiv preprint arXiv:2607.18074*, 2026.

[18] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. NVIDIA tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2018. doi: 10.1109/IPDPSW.2018.00091. arXiv:1803.04014.

[19] Hiroyuki Ootomo and Rio Yokota. Recovering single precision accuracy from tensor cores while surpassing the FP32 theoretical peak performance. *The International Journal of High Performance Computing Applications*, 36(4):475–491, 2022. doi: 10.1177/1094342022 1090256. arXiv:2203.03341.

[20] Anoushka Bhutani, Atul Thakur, Erica Tsai, Kristopher Keipert, and Michele Ceriotti. Unlocking tensor core performance for machine learning interatomic potentials. NVIDIA Technical Blog, 29 October 2026, https://developer.nvidia.com/blog/unlocking-tensor-core-performance-for-machine-learning-interatomic-potentials/, 2026.

[21] NVIDIA. CUDA C++ programming guide: CUDA graphs. Documentation, https://docs.nvidia.com/cuda/cuda-programming-guide/, 2026.

[22] Loup Verlet. Computer "experiments" on classical fluids. I. thermodynamical properties of Lennard-Jones molecules. *Physical Review*, 159(1):98–103, 1967. doi: 10.1103/PhysRev.159.98.

[23] Janosh Riebesell, Rhys E. A. Goodall, Philipp Benner, Yuan Chiang, Bowen Deng, Gerbrand Ceder, Mark Asta, Alpha A. Lee, Anubhav Jain, and Kristin A. Persson. Matbench discovery – a framework to evaluate machine learning crystal stability predictions. *arXiv preprint arXiv:2308.14920*, 2023.

[24] Ilyes Batatia, Chen Lin, Joseph Hart, Elliott Kasoar, Alin M. Elena, Sam Walton Norwood, Thomas Wolf, and Gábor Csányi. Cross learning between electronic structure theories for unifying molecular, surface, and inorganic crystal foundation force fields. *arXiv preprint arXiv:2510.25380*, 2025.

[25] Orion Cohen, Janosh Riebesell, Rhys Goodall, Adeesh Kolluru, Stefano Falletta, Joseph Krause, Jorge Colindres, Gerbrand Ceder, and Abhijeet S. Gangan. TorchSim: An efficient atomistic simulation engine in PyTorch. *AI for Science*, 1(2):025003, 2025. doi: 10.1088/3050-287X/ae1799. arXiv:2508.06628. Software version 0.6.2 used here.

[26] NVIDIA. ALCHEMI toolkit. Software, version 0.2.0 used here. https://github.com/NVIDIA/nvalchemi-toolkit, 2026.

[27] Anders Johansson, Evan Weinberg, Christian R. Trott, Megan J. McCarthy, and Stan G. Moore. LAMMPS-KOKKOS: Performance portable molecular dynamics across exascale architectures. *arXiv preprint arXiv:2508.13523*, 2025.

[28] Dávid Péter Kovács, J. Harry Moore, Nicholas J. Browning, Ilyes Batatia, Joshua T. Horton, Yixuan Pu, Venkat Kapil, William C. Witt, Ioan-Bogdan Magdău, Daniel J. Cole, and Gábor Csányi. MACE-OFF: Short-range transferable machine learning force fields for organic molecules. *Journal of the American Chemical Society*, 147(21):17598–17611, 2025. doi: 10.1021/jacs.4c07099. arXiv:2312.15211.

[29] Ilyes Batatia, William J. Baldwin, Domantas Kuryla, Joseph Hart, Elliott Kasoar, Alin M. Elena, Harry Moore, Mikołaj J. Gawkowski, Benjamin X. Shi, Venkat Kapil, Panagiotis Kourtis, Ioan-Bogdan Magdău, and Gábor Csányi. MACE-POLAR-1: A polarisable electrostatic foundation model for molecular chemistry. *arXiv preprint arXiv:2602.19411*, 2026.

[30] Seung Yul Lee, Hojoon Kim, Yutack Park, Dawoon Jeong, Seungwu Han, Yeonhong Park, and Jae W. Lee. FlashTP: Fused, sparsity-aware tensor product for machine learning interatomic potentials. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 33143–33156, 2025.

[31] William C. Witt. Symmetrix. Software, https://github.com/wcwitt/symmetrix, 2026.

[32] Chuin Wei Tan, Marc L. Descoteaux, Mit Kotak, Gabriel de Miranda Nascimento, Seán R. Kavanagh, Laura Zichi, Menghang Wang, Aadit Saluja, Yizhong R. Hu, Tess Smidt, Anders Johansson, William C. Witt, Boris Kozinsky, and Albert Musaelian. High-performance training and inference for deep equivariant interatomic potentials. *Digital Discovery*, 2026. doi: 10.1039/D5DD00423C. arXiv:2504.16068.

[33] Seán R. Kavanagh, Chuin Wei Tan, Menghang Wang, Marc L. Descoteaux, Gabriel de Miranda Nascimento, Ulrik Unneberg, Laura Zichi, Francesco Libbi, Norma Rivano, Austin Glover, Vivek Bharadwaj, Anders Johansson, William C. Witt, Albert Musaelian, and Boris Kozinsky. Fast and accurate equivariant foundation models for atomistic simulation. *arXiv preprint arXiv:2607.28461*, 2026.

[34] Albert Musaelian, Anders Johansson, Simon Batzner, and Boris Kozinsky. Scaling the leading accuracy of deep equivariant models to biomolecular simulations of realistic size. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23)*, 2023. doi: 10.1145/3581784.3627041. arXiv:2304.10061.

[35] Paul Fuchs, Weilong Chen, Stephan Thaler, and Julija Zavadlav. chemtrain-deploy: A parallel and scalable framework for machine learning potentials in million-atom MD simulations. *arXiv preprint arXiv:2506.04055*, 2025.

[36] Yutack Park, Jaesun Kim, Seungwoo Hwang, and Seungwu Han. Scalable parallel algorithm for graph neural network interatomic potentials in molecular dynamics simulations. *Journal*

of Chemical Theory and Computation, 20(11):4857–4868, 2024. doi: 10.1021/acs.jctc.4c00190. arXiv:2402.03789.

[37] Aidan P. Thompson, H. Metin Aktulga, Richard Berger, Dan S. Bolintineanu, W. Michael Brown, Paul S. Crozier, Pieter J. in 't Veld, Axel Kohlmeyer, Stan G. Moore, Trung Dac Nguyen, Ray Shan, Mark J. Stevens, Julien Tranchida, Christian Trott, and Steven J. Plimpton. LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271:108171, 2022. doi: 10.1016/j.cpc.2021.108171.

[38] Zheyong Fan, Yanzhou Wang, Penghua Ying, Keke Song, Junjie Wang, Yong Wang, Zezhu Zeng, Ke Xu, Eric Lindgren, J. Magnus Rahm, et al. GPUMD: A package for constructing accurate machine-learned potentials and performing highly efficient atomistic simulations. *The Journal of Chemical Physics*, 157(11):114801, 2022. doi: 10.1063/5.0106617. arXiv:2205.10046.

[39] Massimiliano Fasi, Nicholas J. Higham, Mantas Mikaitis, and Srikara Pranesh. Numerical behavior of NVIDIA tensor cores. *PeerJ Computer Science*, 7:e330, 2021. doi: 10.7717/peerj-cs.330.

A Adoption history

Figure 5 shows the recorded step time on the A100 for successive default configurations. It is a chronology of separate jobs, not a paired measurement, and its first bar was taken under the cold protocol.

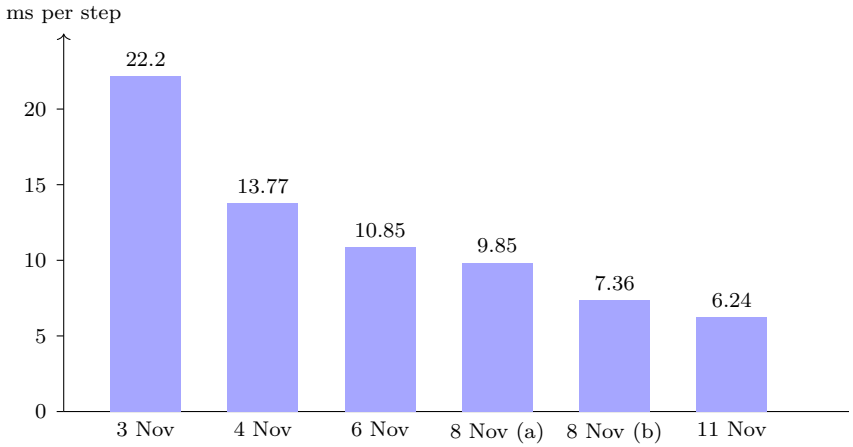


Figure 5: Recorded MD step time on the A100-SXM4-80GB for successive default configurations ($SrTiO_3$, 5,000 atoms). Main changes between bars: custom second-interaction kernels and degree-5 tables (4 Nov); staging and fused first interaction (6 Nov); spline weights, receiver-major forward and composed first-interaction maps (8 Nov; bars a and b are the two sides of that paired study); packed table, generated contraction and hybrid product (11 Nov).

The factor of the acceptance rule. The factor in Eq. (3) was 2 until 7 November 2026. It was raised to 3 after one A100 configuration exceeded the limit in a single quantity ($NbCl_5$ stress, 1.8×10^{-7} against $1.2–1.5 \times 10^{-7}$ eV/Å^3). The current A100 configuration passes that check at a ratio near 1. A threshold changed after a failure is weaker evidence than one fixed in advance, which is one reason the paper reports the errors themselves (Table 3).

B Origin of the mechanisms

Table 14 states where each mechanism came from, as far as the project's records show. Three categories are used: *vendored* code, an *adapted* idea with our own implementation, and *own* work. "Own" describes how a part arose in this project, not priority: where another engine has an equivalent, the table says so. A number of ideas reached the project through research reports written by large language models at the author's request; these are marked. For a few parts the records do not say where the idea came from.

Mechanism	Origin
Receiver-streaming convolution kernels	Vendored: OpenEquivariance pull request 223 [15], after Sobek [17]
Three-product TF32 kernel	Vendored: CUTLASS [10]; method of [19]
Symmetric contraction, remaining tensor operations	Vendored: cuEquivariance [11]
Weights computed inside the convolution; direct spline weights; receiver-major forward	Adapted from Symmetrix-XL [2]
Three-sweep tiled evaluation	Adapted from Symmetrix-XL [2]
Aggregate before projecting (first interaction)	Adapted: the density trick [3]; proposed for this code in an LLM research report
Generated and factored symmetric contraction; shared geometry sums; packed table	Adapted: proposed in LLM research reports
Horner form of the reverse; receiver's adjoint row staged; cell-list screen	Adapted: proposed in LLM-written idea lists and a code review; the cell list is standard
Storing x and s; host state; per-tile neighbour lists	Adapted: outside idea lists; own form
Captured CUDA graphs; Verlet list	Adapted: standard techniques [21, 22]
Chebyshev radial tables and the setup test	Own. Tabulation is established [13]; Symmetrix-XL splines and gates the same networks [2]
Reverse without stored weights; factored reverse	Own, on the vendored kernel body. Symmetrix-XL's reverse also recomputes its radial factors [2]
Fused first-interaction forward and reverse; folded linear maps	Own
Hybrid TF32 / FP16 product	Own
Pipelined step, overlapped rebuild, noise batches, GPU pair search	Own
Wavefront schedule, row reuse, tile-local coordinates	Own
GPU-resident integrator; padded edge capacities; receiver-sorted edges	Not recorded

Table 14: Origin of the mechanisms, from the project's provenance notes. "Own" includes work done in LLM coding sessions directed by the author.

C Records, configurations and software

Checkpoint. The H200 and A100 studies of 9 and 11 November record the same OMAT-0 medium checkpoint, with the SHA-256

`d4b14be9afa294eebdbe31a0280b26a0fa29715771e978cbfd3ca24e0d90307a`

Software. The November studies on the H200, the A100, the RTX 4000 Ada and the RTX 5060 Ti used PyTorch 2.14.1 with CUDA 13.0, Triton 3.8.0, cuEquivariance 0.12.0, OpenEquivariance 0.7.0, mace-torch 0.3.16, e3nn 0.4.4, ASE 3.27.0, CUTLASS 4.8.1 and Python 3.10. Symmetrix-XL was run from its 0.1.1 and 0.1.2 wheels with a LAMMPS plugin built from the matching source, in float32 with its speed profile and a prepared device artifact. The October comparisons of Table 8 deviate:

- **Stock ASE** (24 Oct): the standard environment; model MH-1; a thermal start; one block of 300 steps after 300.
- **TorchSim** (25 Oct): FastMACE in the standard environment, TorchSim in its own with Python 3.12; thermal start, friction 0.01 per fs; FastMACE timed over three blocks of 1,000 steps after 300, TorchSim over three blocks of 200 after 50.
- **ALCHEMI Toolkit** (26 Oct): both engines in one environment with Python 3.12.3, cuEquivariance 0.13.0 and ASE 3.29.0; thermal start, friction 0.01 per fs, skin 0.3 Å; four blocks of 300 steps after 300. The FastMACE revision is not recorded.

Deviations from the paired protocol. The paired protocol of Section 8.1 is 200 warm-up steps and six alternating blocks of 256 steps in one process. The following studies used another schedule:

- **Radial tables, first evidence** (Table 5, first row): eight blocks of 200 steps.
- **H200, $NbCl_5$** (Table 4): eight blocks of 256 steps, the baseline side of a separate study at revision `e3049399`; the H200 value for $SrTiO_3$ is from `5c5ade67`.
- **Cell-list screen** (Section 8.6; the 20,480- and 40,000-atom rows of Table 9): six blocks of 128 steps.
- **69,120 atoms** (Table 9): not paired. Separate processes, 600 timed steps after 100, in the order ordered, shuffled, shuffled, ordered; the table gives the two ordered runs.
- **214,375 atoms and the whole-system capacity rows** (Tables 9 and 10): one process, 200 timed steps after 20.
- **MACE-OFF23** (Table 11): eight blocks of 1,000 steps after 500 warm-up steps. MACE-POLAR-1 used the standard schedule.
- **OMAT-0 small** (Tables 7 and 11): not paired. One process from a thermal start, 200 timed steps after 100.
- **Tiled path** (Table 6, and the device-state rows of Table 10): alternating blocks of force calls, not MD steps; four blocks of two calls per side for the captured graphs, from which the A100's 2.04 s comes. The RTX 5060 Ti's 6.1 s is the steady call of one profile run. The host-state rows are one compiled call after one eager call.

Records. Table 15 names the record behind each table and figure. A record is a directory of the project repository that holds the job scripts, logs and results of one study.

Item	Record	Revision	Protocol
Profile (Table 1)	step-limiters-20261109	c520e074, 0df22417	counter profile
Accuracy (Table 3)	sc0-op-20261109, hybrid-gemm-20261111	5c5ade67, 4195be0d	18 cells
Step times (Table 4)	sc0-op-20261109, hybrid-gemm-20261111, ada-5060-screen-20261110, step-limiters-20261109	5c5ade67, 4195be0d, edfc0115, e3049399	paired; see above
Paired ratios (Tables 5, 6)	provenance note; tiled-speed-20261110, tiled-speed-20261111	several	paired; see above
Symmetrix-XL runs (Table 7)	loup-a100 (note), symmetrix-xl-h200-loup-20261105, sx012-profile-20261106, symmetrix-xl-h200-omat0-small-20261109, vram-compare-rtx5060ti-20261105, nacl-compare-rtx5060ti-20261105	7430fb51, d599cc70 (FastMACE, 3 and 6 Nov)	cold; thermal start; one mixed
Stages (Figure 4)	sx012-profile-20261106, step-limiters-20261109	d599cc70	profiles
General engines (Table 8)	phase-4 note; TorchSim and ALCHEMI comparison notes	4c574d2, 5a3166c; see above	see above
Size (Table 9)	ada-5060-screen-20261110, binned-screen-5060ti-20261111, shuffled-order-5060ti-20261111, rung-b-20261109	b951e804, f31d4f50, cd56800c	four schedules; see above
Capacity, whole system (Table 10)	rung-b-20261109	cd56800c	MD, 200 steps after 20
Capacity, device state	tiled-speed-20261111	d7d9e7ce, d4014c75	A100: paired; RTX 5060 Ti: one profile run
Capacity, host state	tiled-capacity-20261109	d036568b, 7e4fbf4c	one compiled call after one eager call
Other models (Table 11)	off-polar-a100-20261108, h200-omat0-small-20261108	b4ec7053, e2206bb6	three schedules; see above
Radial tables (Section 5.1)	packed-table-lab-20261109, first-interaction-20261111	–	offline

Table 15: Records behind the tables and figures. Revisions are abbreviated commit identifiers of the FastMACE repository; a dash means the record names several.